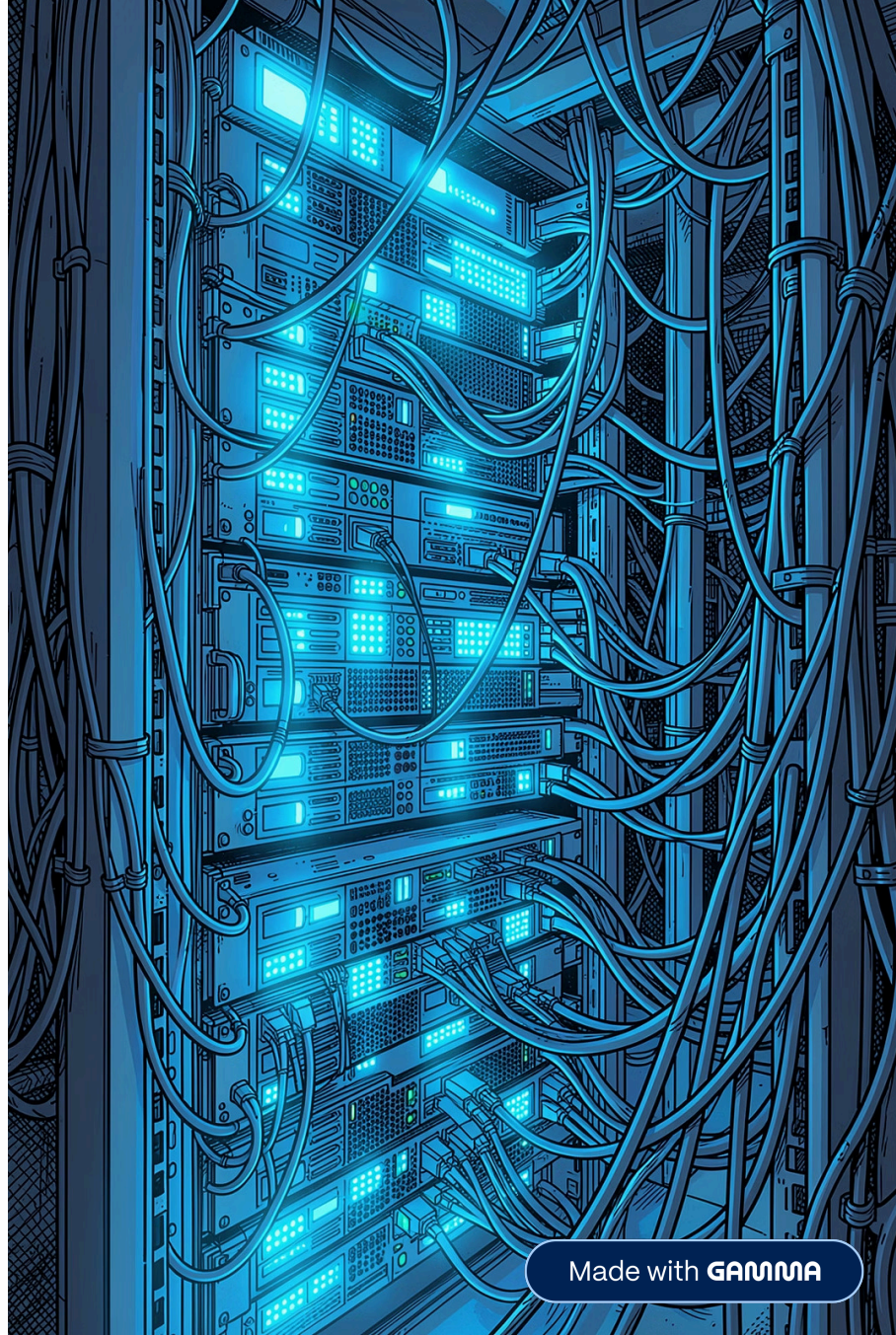
BACKEND ENGINEERING

INFOSEC STUDENTS

Containerization & Docker Fundamentals

A practical guide to reproducible environments, modern deployment workflows, and backend engineering practices — built for developers who need things to work the same way everywhere.



Made with GAMMA

The Problem Before Docker

Classic Scenario

Developer A: ✓ **Works on my machine.**
Developer B: ✗ **Cannot run it.**

Common Causes

- Different OS versions
- Mismatched language runtimes (e.g., Go 1.20 vs 1.24)
- Missing or conflicting library versions
- Undocumented local configuration

How can we make software run consistently everywhere?

Made with GAMMA

What Is a Container?

A container packages everything your application needs to run — eliminating environment mismatch.

Application Code

Your compiled binary or source

Runtime

Go, Node, Python — exact version

Dependencies

Libraries and system packages

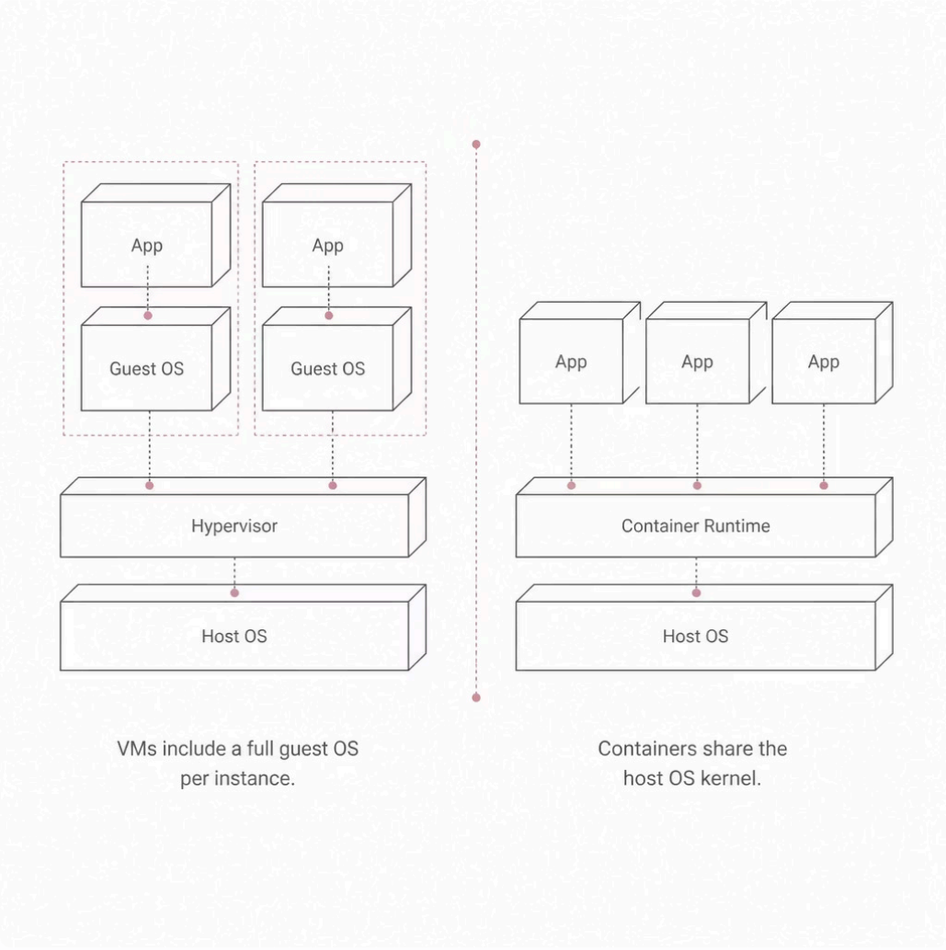
Configuration

Environment variables, ports

Build once. Run anywhere. Same behavior on your laptop, CI, and production.

Made with GAMMA

Virtual Machine vs. Container



Why Containers Win for Backend Services

Containers share the host OS kernel, making them lightweight and ideal for microservices.

Smaller

MBs vs GBs — no guest OS overhead

Faster

Seconds to start, not minutes

Portable

Same image runs on any host with Docker

Made with GAMMA

What Is Docker?

Docker is the most widely adopted container platform — the standard for packaging and shipping applications.

1. **Code**
   Write your application
2. **Dockerfile**
   Define build instructions
3. **Image**
   Immutable, shareable artifact
4. **Container**
   Running instance

Made with GAMMA

Docker Image vs. Container

🗂️ Image — The Blueprint

- Read-only template
- Built from a Dockerfile
- Reusable across environments
- Stored in a registry (e.g., Docker Hub)

Think of it as a **class definition**.

▶️ Container — The Running Instance

- Live, executing process
- Has state (memory, filesystem writes)
- Can start, stop, pause, restart
- Isolated from host and other containers

Think of it as an **object instantiated** from that class.

ⓘ One image → many containers

Made with GAMMA

Building Your First Dockerfile

```
FROM golang:1.24
WORKDIR /app
COPY . .
RUN go build -o mini-asm .
CMD ["./mini-asm"]
```

Line by Line

01

FROM

Base image — official Go 1.24

02

WORKDIR

Sets working directory inside container

03

COPY + RUN

Copy source, compile binary

04

CMD

Process that runs when container starts

Made with GAMMA

Docker Build & Run

1	2	3
Build the Image	Run the Container	Verify It Works
`docker build -t mini-asm .`	`docker run -p 8080:8080 mini-asm`	`curl http://localhost:8080/health`
Reads Dockerfile, produces a tagged image.	Starts a container, maps port 8080 to host.	Should return `200 OK` — same as local.

Containerized apps should behave **exactly** like local ones.

Made with GAMMA

Why Docker Compose?

Real applications are never a single process. A typical backend stack includes:

Managing each container manually with `docker run` is error-prone. Docker Compose defines all services in one file and starts them together.

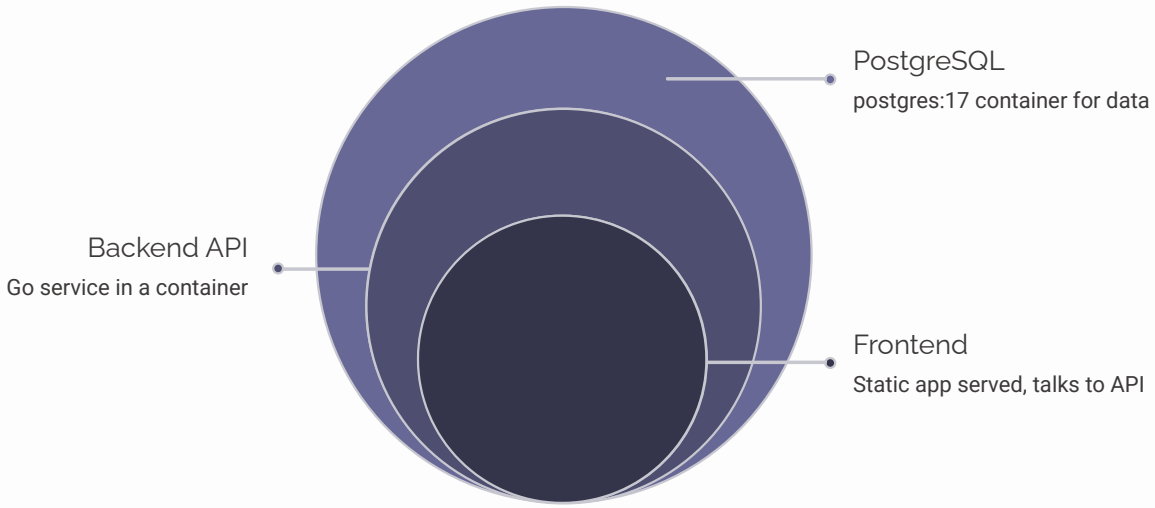
```
services:
  api:
    build: .
    ports:
      - "8080:8080"
  postgres:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: secret
```

One command: `docker compose up`

API	Database	Cache	Message Queue

Made with GAMMA

Containerizing Mini ASM



Mini ASM's architecture: frontend talks to a Go API container, which connects to a PostgreSQL container — all started with one command.

Today

`api` + `postgres` services in `docker-compose.yml`

Next

Add `redis` for caching, `scanner-worker` for async tasks

Key Takeaway

Containers standardize development and deployment — eliminating environment drift.